

Murilo Tomaz Gonzaga

Prof. Douglas Rodrigues

Métodos Numéricos Computacionais

01 de maio de 2025

## **Relatório do Trabalho prático 1 - Zero de Função**

### **Implementação:**

Foi utilizada a linguagem de programação Python para implementar a ideia do programa. Entre suas inúmeras bibliotecas, utilizei **matplotlib**, **math**, **time** e **os**, que deram suporte a toda a construção do projeto, desde a implementação da interface gráfica até a facilitação na implementação das funções.

A principal ideia do projeto era implementar os métodos de busca de raízes de funções aprendidos em aula. Ou seja, foram implementados os métodos da bissecção, falsa posição e Newton-Raphson, que têm como objetivo calcular as raízes aproximadas de uma função, retornando também o número de iterações, o tempo de execução e o erro. Como solicitado na folha de requisição do projeto, todas essas implementações são gerenciadas por uma interface no CMD, com opções por meio de linhas de comando e escolha do usuário.

*Imports e variáveis globais:*

```
import math
import os
import timeit
import matplotlib.pyplot as plt
import numpy as np

#valores globais de epsilon e sigma, utilizados no problema 4
epsilon = 1.0
sigma = 1.0
```

Como dito anteriormente, *import* de bibliotecas python que foram extremamente úteis para a resolução do trabalho e além disso temos duas variáveis flutuantes globais que são definidas para que possamos utilizar no problema em que são requisitadas.

### *Funções numéricas:*

```
# Funções numéricas
def problema1(x):
    return 5 * x - 100

def problema2(V):
    return 1e-12 * (math.exp(V / 0.025) - 1) - 0.01

def problema3(x):
    return (x**2 / 1e5 - x) - 1e-5

def problema4(r):
    return 24 * epsilon * (2 * (sigma / r) ** 13 - (sigma / r) ** 7)

def problema5(K):
    return K - 1 - math.exp(-K)
```

Nessa parte do código são realizadas a transcrição das funções descritas nos problemas para o modelo codificado.

### *Métodos computacionais:*

*Em todos os métodos a seguir, além do algoritmo padrão é possível visualizar um vetor chamado pontos onde são guardados os pontos com a forma de um histórico para que seja possível a feição do gráfico mais à frente.*

```
#bissecacao
def bissecacao(f, a, b, E=1e-4, max_i=1000):
    pontos = []
    for i in range(max_i):
        x = (a + b) / 2
        fx = f(x)
        pontos.append((i + 1, x, fx))
        if abs(fx) < E or abs(b - a) < E:
            return x, pontos
        if f(a) * fx < 0:
            b = x
        else:
            a = x
    return None, pontos
```

Implementação do método da bisseção em python, seguindo as instruções do pseudocode presentes nos slides das aulas.

```

#falsa posicao
def falsa_posicao(f, a, b, E=1e-4, max_i=1000):
    pontos = []
    if f(a) * f(b) ≥ 0:
        return None, pontos
    for i in range(max_i):
        fa = f(a)
        fb = f(b)
        x = (a * fb - b * fa) / (fb - fa)
        fx = f(x)
        pontos.append((i + 1, x, fx))
        if abs(fx) < E or abs(b - a) < E:
            return x, pontos
        if fa * fx < 0:
            b = x
        else:
            a = x
    return None, pontos

```

Implementação do método da falsa posição em python, seguindo as instruções do pseudocode presentes nos slides das aulas.

```

#newton-raphson
def newton(f, df, x0, E=1e-4, max_i=1000):
    pontos = []
    for i in range(max_i):
        fx = f(x0)
        dfx = df(x0)
        if abs(dfx) < 1e-10:
            return None, pontos
        x1 = x0 - fx / dfx
        pontos.append((i + 1, x1, f(x1)))
        if abs(f(x1)) < E or abs(x1 - x0) / max(1, abs(x1)) < E:
            return x1, pontos
        x0 = x1
    return None, pontos

# Derivada por diferenca finita central
def derivada_numerica(f, x, h=1e-6):
    return (f(x + h) - f(x - h)) / (2 * h)

```

Implementação do método de Newton-Raphson em python, seguindo as instruções do pseudocode presentes nos slides das aulas, em seguida a implementação do método da diferença finita, pois é necessário a derivada no método de newton, utilizamos a diferença central com h incrivelmente pequeno, fazendo-se necessária somente uma iteração.

### *Funções utilitárias:*

```
# Funções auxiliares
def clear_terminal():
    os.system("cls" if os.name == "nt" else "clear")

def medir_tempo(func): #funcao para medir o tempo de cada metodo
    inicio = timeit.default_timer()
    resultado = func()
    return resultado, timeit.default_timer() - inicio

def executar_metodo(nome, metodo, args, func, cor): #execucao do metodo de retorno
    das info para que o vet metodo na main consigo passar para as "show" functions
    (raiz, pontos), tempo = medir_tempo(lambda: metodo(*args))
    if pontos:
        return (nome, pontos, raiz, cor, tempo, len(pontos))
    return None
```

Nessa seção são implementadas, funções auxiliares, ou seja, funções que serão utilizadas por outras funções no futuro. Primeiramente temos a função que limpa o terminal realizando o comando no mesmo, seja ele terminal do windows ou unix, a seguir a função que realiza a medição de tempo, usada futuramente para medir o tempo de execução de um método.

### *Funções ilustrativas:*

```
#funcao graficas
def plot_results(metodos, titulo):
    plt.figure(figsize=(15, 5))
    nomes = [m[0] for m in metodos]
    tempos = [m[4] for m in metodos]
    iteracoes = [m[5] for m in metodos]

    plt.subplot(1, 3, 1)
    plt.bar(nomes, tempos, color=["blue", "black", "red"])
    plt.title("Tempo de Execução (s)")
    plt.ylabel("Segundos")

    plt.subplot(1, 3, 2)
    plt.bar(nomes, iteracoes, color=["blue", "black", "red"])
    plt.title("Número de Iterações")
    plt.ylabel("Iterações")

    plt.subplot(1, 3, 3)
    for nome, pontos, cor in metodos:
        i, x, _ = zip(*pontos)
        plt.plot(i, x, "-", color=cor, label=nome)
    plt.xlabel("Iteração")
    plt.ylabel("Valor de x")
    plt.title(f"Convergência da Raiz - {titulo}")
    plt.legend()
    plt.grid(True)

    plt.tight_layout()
    plt.savefig(f"Análise de performance - {titulo}.png")
    plt.close()
```

```
#impressao de um relatorio no cmd
def relatorio(metodos, func):
    print("\nRELATÓRIO DE RESULTADOS:\n")
    for nome, pontos, raiz, _, tempo, iteracoes in metodos:
        convergiu = "Sim" if raiz is not None else "Não"
        erro = abs(func(raiz)) if raiz is not None else "-"
        print(
            f"[{nome}]\n - Convergiu: {convergiu}\n - Iterações: {iteracoes}\n - "
            f"Tempo: {tempo:.6f} segundos\n - Raiz: {raiz if raiz is not None else '-'}\n "
            f"- Erro absoluto |f(x)|: {erro if erro != '-' else f'{erro:.4e}'}\n"
        )
```

Primeiramente, utiliza-se a biblioteca **Matplotlib** para gerar representações gráficas das informações, com a criação de três gráficos: o primeiro exibe o tempo de execução de cada método aplicado ao problema; o segundo mostra o número de iterações de cada método; e o terceiro apresenta a convergência da raiz em relação ao

número de iterações, permitindo observar a variação das raízes ao longo do processo iterativo.

Na segunda função, é gerado um relatório no terminal (cmd) com as informações obtidas pelos métodos, apresentado de forma detalhada para uma análise mais precisa do desempenho.

```
#escolha de um problema
def escolher_problema():
    global epsilon, sigma
    print("\nProblemas disponíveis:")
    for i in range(1, 6):
        print(f"{i} - Problema {i}")
    escolha = int(input("Escolha o problema: "))
    if escolha == 4:
        epsilon = float(input("Digite o valor de epsilon (padrão=1.0): ") or 1.0)
        sigma = float(input("Digite o valor de sigma (padrão=1.0): ") or 1.0)
    return escolha
```

A terceira função ilustrativa porém também interativa, é a realização de um menu com linhas de comando como pedido no pdf para realização do projeto, ou seja, temos a opção de selecionar qual problema será realizado.

*Main:*

```

#main function
def main():
    escolha = escolher_problema() #escolha do problema
    func = funcoes_numericas[escolha - 1]#vetor para pegar as funcoes de cada
    problema

    a = float(input("a = "))
    b = float(input("b = "))
    x0 = float(input("x (Newton) = "))
    E = float(input("Digite o valor de erro (ex: 1e-3): ") or 1e-4)

    clear_terminal()

    metodos = [] #vetor para guardar a informacao de cada metodo

    configs = [
        ("Bisseção", bissecao, (func, a, b, E), "blue"),
        ("Falsa Posição", falsa_posicao, (func, a, b, E), "black"),
        ("Newton", newton, (func, lambda x: derivada_numerica(func, x), x0, E),
        "red"),
    ]

    for nome, metodo, args, cor in configs: #iterar em configs executando os metodos
        resultado = executar_metodo(nome, metodo, args, func, cor)
        if resultado:
            metodos.append(resultado) #adiciona para realizar as funcoes de "show"

    titulo = f"Problema {escolha}"
    if metodos:
        plot_results(metodos, titulo)
        relatorio(metodos, func)
    else:
        print("Nenhum método convergiu para os parâmetros fornecidos.")

```

A função **main** é responsável por organizar a lógica geral do programa, integrando as demais funções e utilizando funções auxiliares previamente definidas. Nela, o problema é recebido por meio da interface de linha de comando. Para cada método numérico estabelecido, é criado um vetor correspondente, conforme o problema em questão. Também são recebidos os parâmetros necessários para cada método iterativo, incluindo, quando aplicável, o erro desejado. Em seguida, os métodos são executados, e os parâmetros e resultados são repassados para as funções ilustrativas, que se encarregam de gerar as representações gráficas.

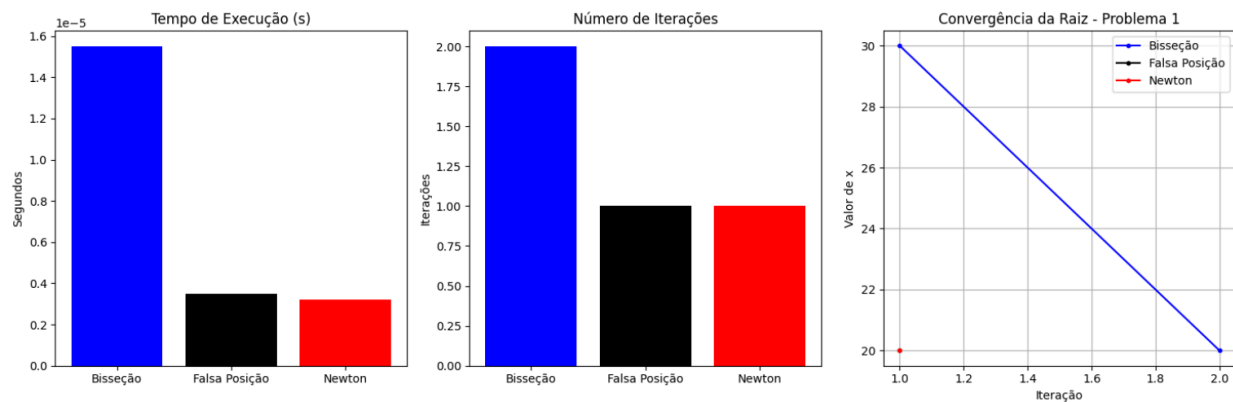
*Execução:*

```
while True:
    clear_terminal()
    main()
    if input("Deseja resolver outro problema? (s/n): ").lower() != "s":
        break
```

Em um while infinito, realizamos o reprocessamento para que seja possível a re-utilização do programa em outros problemas sem precisar rodar novamente, apenas digitando “s” para continuar.

## Análises Gráficas:

### Problema 1:



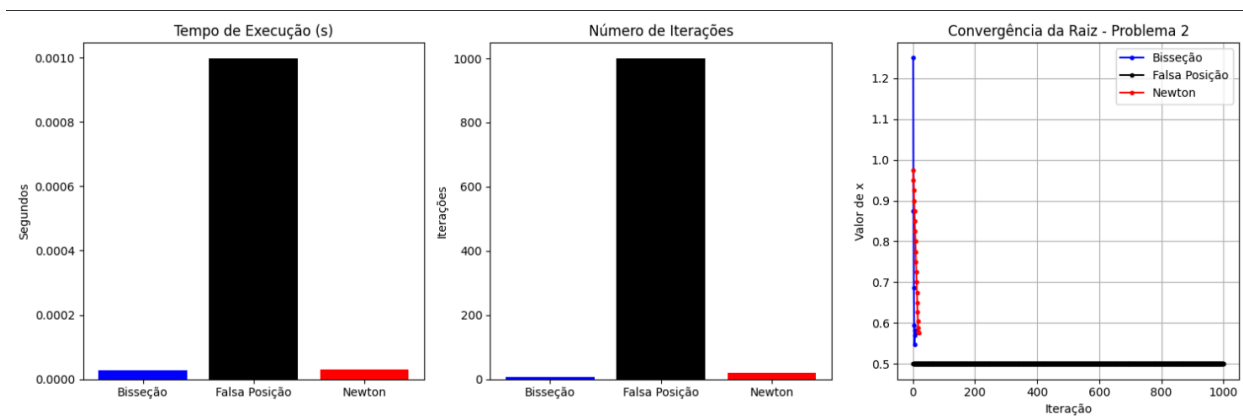
```
[Bisseção]
- Convergiu: Sim
- Iterações: 19
- Tempo: 0.000037 segundos
- Raiz: 20.000019073486328
- Erro absoluto |f(x)|: 9.5367e-05

[Falsa Posição]
- Convergiu: Sim
- Iterações: 1
- Tempo: 0.000003 segundos
- Raiz: 20.0
- Erro absoluto |f(x)|: 0.0000e+00

[Newton]
- Convergiu: Sim
- Iterações: 1
- Tempo: 0.000004 segundos
- Raiz: 19.99999999396914
- Erro absoluto |f(x)|: 3.0154e-08
```

Neste primeiro problema, foram utilizados os seguintes valores:  $a = 10$ ,  $b = 40$ ,  $x = 1$  e  $\epsilon = 1e-4$ . Ao analisar as informações brutas e os gráficos gerados, observamos que os métodos da bissecção e da falsa posição atingiram a raiz exata, enquanto o método de Newton, devido ao valor do erro estabelecido, chegou extremamente próximo da solução correta. Também foi possível notar que o número de iterações e o tempo de execução do método da bissecção foram significativamente maiores para essa combinação de valores, o que indica que, do ponto de vista computacional, o método da falsa posição seria a escolha mais eficiente.

## Problema 2:



### RELATÓRIO DE RESULTADOS:

#### [Bisseção]

- Convergiu: Sim
- Iterações: 8
- Tempo: 0.000029 segundos
- Raiz: 0.576171875
- Erro absoluto  $|f(x)|$ :  $2.1247e-04$

#### [Falsa Posição]

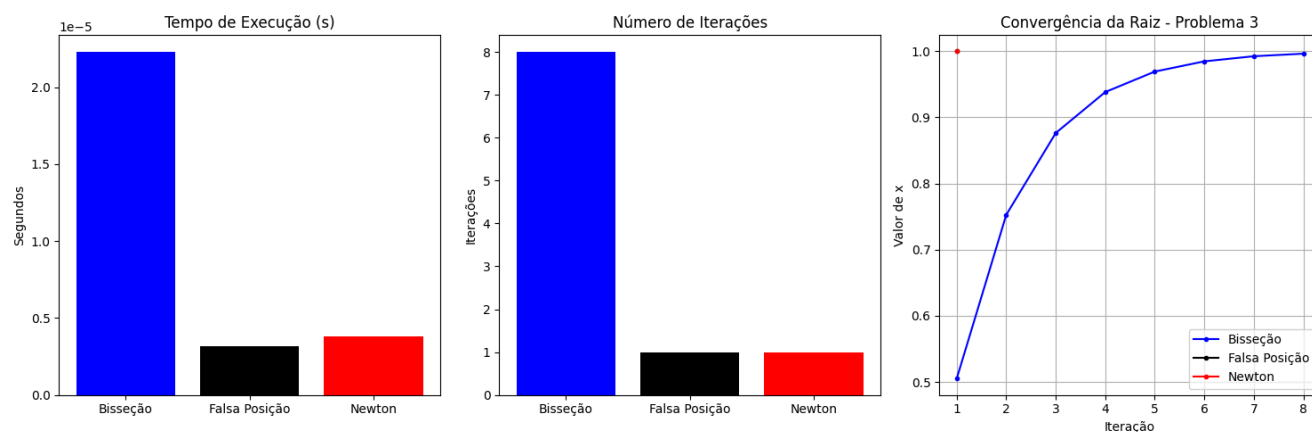
- Convergiu: Não
- Iterações: 1000
- Tempo: 0.000997 segundos
- Raiz: -
- Erro absoluto  $|f(x)|$ : -

#### [Newton]

- Convergiu: Sim
- Iterações: 19
- Tempo: 0.000032 segundos
- Raiz: 0.57577199840405
- Erro absoluto  $|f(x)|$ :  $5.0417e-05$

Neste problema, foram utilizados os seguintes valores:  $a = 0.5$ ,  $b = 2$ ,  $x = 1$  e  $\epsilon = 1e-4$ . Neste problema, a raiz encontrada foi aproximadamente 0,576, e apenas dois métodos conseguiram convergir. O método da falsa posição não obteve sucesso na convergência devido à natureza da função e aos valores atribuídos para a operação. Como resultado, ele realizou mil iterações sem alcançar um valor aceitável, o que inflacionou significativamente seus dois gráficos (tempo e número de iterações). É interessante destacar que o método de Newton apresentou o menor número de iterações até alcançar o resultado, o que demonstra que, para este problema específico e com essa combinação de valores, ele se mostra a opção mais viável do ponto de vista computacional.

### Problema 3:



**RELATÓRIO DE RESULTADOS:****[Bisseção]**

- Convergiu: Sim
- Iterações: 8
- Tempo: 0.000022 segundos
- Raiz: 0.9961328125
- Erro absoluto  $|f(x)|$ : 7.7095e-08

**[Falsa Posição]**

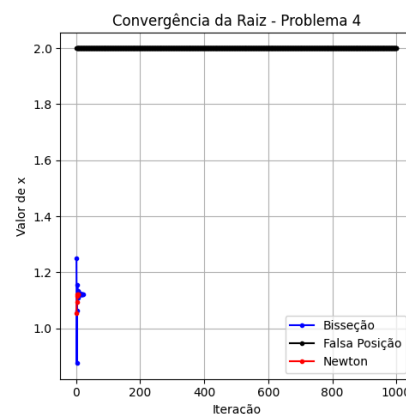
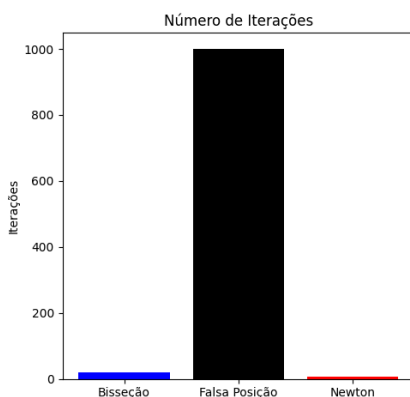
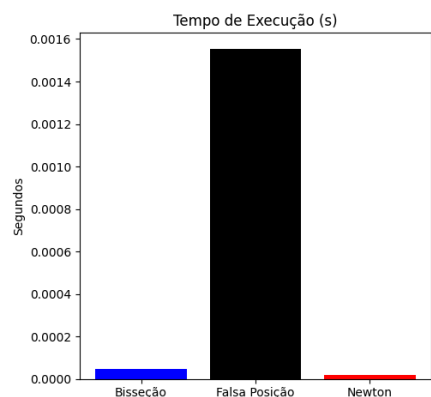
- Convergiu: Sim
- Iterações: 1
- Tempo: 0.000003 segundos
- Raiz: 0.9999900990099109
- Erro absoluto  $|f(x)|$ : 9.8021e-11

**[Newton]**

- Convergiu: Sim
- Iterações: 1
- Tempo: 0.000004 segundos
- Raiz: 0.9999950000250001
- Erro absoluto  $|f(x)|$ : 2.5000e-16

Neste problema, foram utilizados os seguintes valores:  $a = 0.01$ ,  $b = 1$ ,  $x = 0.5$  e  $\epsilon = 1e-6$ . Todos os métodos convergiram para a mesma raiz real. Sabemos que a equação possui duas raízes reais — uma positiva e outra negativa — e, devido aos valores iniciais arbitrários escolhidos, acabamos caindo apenas na raiz positiva. Nesse cenário, o método da falsa posição apresentou melhor desempenho em termos de tempo de execução e número de iterações. No entanto, é importante ressaltar que isso se deve às condições iniciais fornecidas. O método de Newton, por sua vez, também se mostrou eficaz e teria grande utilidade, uma vez que fornece resultados precisos.

### Problema 4:



### RELATÓRIO DE RESULTADOS:

#### [Bisseção]

- Convergiu: Sim
- Iterações: 21
- Tempo: 0.000046 segundos
- Raiz: 1.1224620342254639
- Erro absoluto  $|f(x)|$ : 8.0485e-07

#### [Falsa Posição]

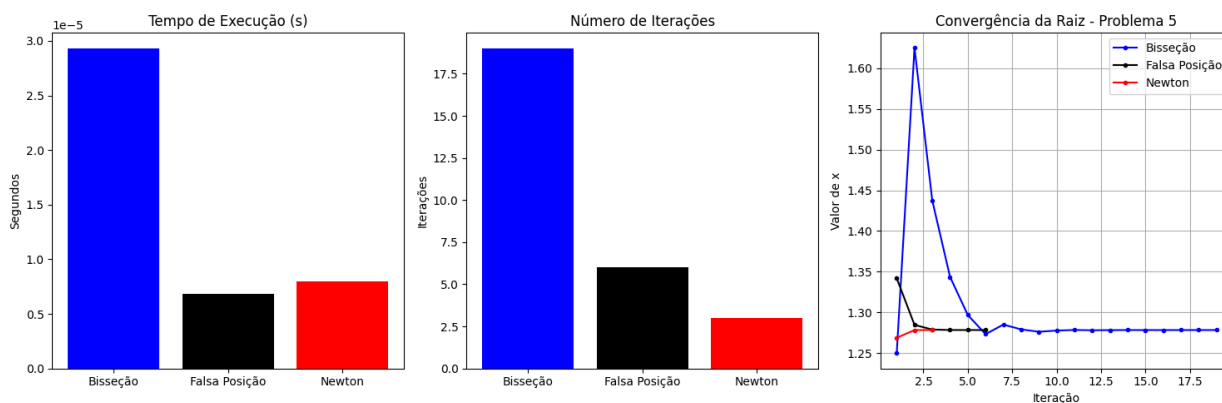
- Convergiu: Não
- Iterações: 1000
- Tempo: 0.001553 segundos
- Raiz: -
- Erro absoluto  $|f(x)|$ : -

#### [Newton]

- Convergiu: Sim
- Iterações: 6
- Tempo: 0.000020 segundos
- Raiz: 1.1224620482960928
- Erro absoluto  $|f(x)|$ : 7.5892e-10

Neste problema, foram utilizados os seguintes valores:  $a = 0.5$ ,  $b = 2$ ,  $x = 1$  e  $\epsilon = 1e-6$ . O método de falsa posição novamente não convergiu por conta da natureza da função e dos valores dados por nós em suas variáveis de método. Porém, se analisarmos entre os que convergiram, vemos que o de Newton sai com um pouco de vantagem, pois realizou menos iterações e acabou sendo metade do tempo de computação do da bisseção. Porém, ambos chegaram na mesma raiz com pouquíssima diferença.

### Problema 5:



### RELATÓRIO DE RESULTADOS:

#### [Bisseção]

- Convergiu: Sim
- Iterações: 19
- Tempo: 0.000029 segundos
- Raiz: 1.2784643173217773
- Erro absoluto  $|f(x)|$ : 2.8822e-07

#### [Falsa Posição]

- Convergiu: Sim
- Iterações: 6
- Tempo: 0.000007 segundos
- Raiz: 1.278465188571206
- Erro absoluto  $|f(x)|$ : 8.2565e-07

#### [Newton]

- Convergiu: Sim
- Iterações: 3
- Tempo: 0.000008 segundos
- Raiz: 1.2784645427503598
- Erro absoluto  $|f(x)|$ : 1.3697e-11

Neste problema, foram utilizados os seguintes valores:  $a = 0.5$ ,  $b = 2$ ,  $x = 1$  e  $\epsilon = 1e-6$ . Fazendo uma análise gráfica básica, percebemos que o método da bisseção acabou sendo menos viável, pois precisou fazer muitas repetições e seu tempo de execução acabou sendo maior que o dos demais. Portanto, para esses valores e problema, o método mais viável acaba sendo disputado entre o de Newton, que teve menos iterações, mas demorou mais tempo computacional, e o da falsa posição, que foi mais rápido, porém precisou de mais iterações

## CONCLUSÃO

Com a realização deste trabalho, ficou nítida a importância dos métodos computacionais aprendidos em sala de aula. A utilização e implementação desses métodos representam desafios, mas, com o algoritmo estruturado e a ideia bem compreendida, o processo se torna mais acessível. Além disso, a proposta de aplicar diversos algoritmos a diferentes funções e problemas torna a prática mais interessante e contribui significativamente para o aprendizado, incentivando uma imersão profunda na linguagem Python.